

A Prompt Is the Complete Model Input

A prompt is all information supplied to the model for one generation.



CORE CONCEPT

Message Roles Separate Sources of Authority

Chat APIs represent context as ordered messages. Exact role support and precedence depend on the provider. Roles help separate content, but applications must still enforce authorization in code. Untrusted text inside a retrieved document does not become a trusted system instruction.

System/developer instruction: application-owned behaviour and boundaries

User message: requested task and user-provided content

Assistant message: prior model output

Tool message: result returned by an external operation

RUN IT AND INSPECT THE RESULT

Good Instructions Define Task, Evidence, and Boundaries



Extract explicit action items from TRANSCRIPT.
Do not infer missing owners or dates.
Return objects matching the supplied schema.

OUTPUT

Structured action items or an empty list when none are explicit.

This is testable against labelled examples.

REASONING SEQUENCE

Examples Demonstrate the Required Pattern

Frame

field names and types;

1**Transform**

how ambiguous cases are handled;

2**Validate**

desired level of detail;

3**Explain**

use of null, empty list, or refusal;

4

Input: Call Mina tomorrow. Output: {"action":"Call Mina","date":null} Input: Send report by Friday. Output: {"action":"Send report","date":"Friday"} The first example teaches not to invent a date.

CORE CONCEPT

Prompt Injection Treats Data as Instructions

Prompt injection occurs when untrusted content tries to alter model behaviour. Direct injection comes from a user. Indirect injection is embedded in retrieved files, websites, emails, or tool results. Defences include:

keep trusted instructions separate;

minimize secrets available to the model;

validate output;

authorize every tool action in code;

restrict tool permissions and arguments;

require human approval for high-impact actions;

RUN IT AND INSPECT THE RESULT

A JSON Schema Defines Machine-Checkable Output

```
class ActionItem(BaseModel):  
    action: str  
    owner: str | None  
    priority: Literal["low", "medium", "high"]
```

OUTPUT

Valid JSON becomes a typed ActionItem; wrong or missing fields raise validation errors.

`{"action":"Send report","owner":null,"priority":"high"}` is structurally valid.

CORE CONCEPT

Parse, Validate, and Handle Failure Explicitly

For every generated object: Retries should be limited and used only when another attempt can reasonably fix the failure. Preserve the original failure for debugging.

receive provider response;

parse the JSON or structured object;

validate against the schema;

validate domain rules;

check evidence;

store or act only after all checks pass.

REASONING SEQUENCE

A Tool Definition Is a Proposed Function Contract

Frame

name;

1

Transform

purpose;

2

Validate

argument schema;

3

Explain

required fields;

4

Tool: getweather(city: string, date: ISO date) Proposed call: {"city":"Colombo","date":"2026-07-29"} The application validates date format and allowed service scope before execution.

RUN IT AND INSPECT THE RESULT

Tool Calling Is a Multi-Step Conversation

```
arguments = ToolArgs.model_validate(tool_call.arguments)
authorize(user, tool_call.name, arguments)
result = execute(tool_call.name, arguments)
```

OUTPUT

A validated, authorized tool result or a controlled error.

The model proposes `getorder(orderid="A123")`. The application confirms the user owns A123, calls a read-only service, then returns the timestamped status to the model.

QUALITY GATE

Structured Extraction Still Needs Evidence Checks



title;



date;



time;



location;

QUALITY GATE

Prompt Quality Requires a Test Dataset



normal cases;



missing fields;



conflicting statements;



long inputs;

GUIDED LAB

Guided Lab: Build a Validated Extraction Flow

01

**separates trusted
instructions from
transcript text**

02

**defines a typed
output schema**

03

**preserves null for
absent fields**

04

**includes one
zero-shot and one
few-shot version**